МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине

«Облачные и туманные вычисления»

Выполнил:
Студент группы P3411
Болорболд Аригуун

Преподаватель:
Перл Ольга Вячеславовна

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 4
   2.1. Приложение 4
   2.2. Ход работы 6
3. Вывод 11

1. Текст задания

Цель:
Изучить разницу между простым запуском контейнера и оркестрацией сервиса. Эта работа закрывает пробел между «работает на моем ноутбуке» и «работает в кластере».

Сценарий:
Вы выступаете в роли DevOps-инженеров простого интернет-магазина, которому нужно справиться с наплывом трафика (например, в «Черную пятницу»).

Приложение:

- **Frontend**: Веб-страница на `Python/Flask`, отображающая счётчик посещений.
- **Backend**: База данных `Redis` для хранения количества хитов.

Ход работы:

1. **Контейнеризация**: Написать `Dockerfile` для `Python`-приложения
2. **Docker Compose**: Написать `docker-compose.yml` для локального запуска связки `App + Redis`. Убедиться, что они «видят» друг друга.
3. **Инициализация** `Swarm`: Перевести `Docker` в режим `Swarm` (`docker swarm init`).
4. **Развертывание стека** (`Deploy Stack`): Конвертировать файл `compose` в деплой стека (`Stack Deploy`).
5. **Тест на отказ** (`Self-healing`): Вы вручную «убиваете» (`docker kill`) активный контейнер и наблюдаете, как `Swarm` автоматически пересоздаёт его.
6. **Масштабирование**: Выполнить команду `docker service scale frontend=5`.

Ключевые знания: Обнаружение сервисов (`Service Discovery`) — как приложение находит находит `Redis` по имени; реплики и самовосстанавливающиеся системы.

2. Выполнение

2.1. Приложение

2.1.1. Frontend

```
import os
from flask import Flask, jsonify
import redis

redis_client = redis.Redis(
    host=os.getenv("REDIS_HOST", "redis"),
    port=int(os.getenv("REDIS_PORT", "6379")),
    db=int(os.getenv("REDIS_DB", "0")),
    password=os.getenv("REDIS_PASSWORD"),
    decode_responses=True
)

app = Flask(__name__)

HTML_TEMPLATE = """

<!DOCTYPE html>
<html lang="mn">
<head>
    <meta charset="UTF-8">
    <title>Онлайн магазин - ЛР№1</title>
    <style>
    # Здесь находится CSS, для содержимости отчёта его убрал
    </style>
</head>
<body>

    <nav>
        <a href="#">Домашняя страница</a>
        <a href="#">Продукты</a>
        <a href="#">Про нас</a>
        <a href="#">Корзина (0)</a>
    </nav>

    <div class="container">
        <p style="color: #666;">Число посещений:</p>
        <div class="counter">{{ count }}</div>

        <div class="products-grid">
            <div class="product-card">
                <img src="https://i01.appmifile.com/webfile/
```

```
globalimg/products/pc/mi-smart-band-5-zc/Spec_PC2560_03.jpg"
alt="Продукт">
                <h3>Смарт-часы</h3>
                <p>9,000₽</p>
                <button class="button">В корзину</button>
            </div>
            <div class="product-card">
                <img src="https://lunafon.ru/image/cache/webp/
catalog/Accessories/jbl-tune-760nc-black-1000x1000.webp"
alt="Продукт">
                <h3>Наушники</h3>
                <p>4,500₽</p>
                <button class="button">В корзину</button>
            </div>
            <div class="product-card">
                <img src="https://cdn.werfstore.ru/wp-content/
uploads/2021/05/65.png" alt="Продукт">
                <h3>Рюкзак</h3>
                <p>7,500₽</p>
                <button class="button">В корзину</button>
            </div>
        </div>

        <div style="margin-top: 30px;">
            <a href="/" class="button">🔄</a>
            <a href="/api" class="button">📊</a>
        </div>

        <div class="version">Версия: {{ version }}</div>
    </div>
    <footer>
        &copy; Болорболд Аригуун P3411 | XVIIStarPt__ | 2026.
    </footer>
</body>
</html>
"""

@app.route("/")
def index():
    count = redis_client.incr("hits")
    version = os.getenv("APP_VERSION", "v1")
    return render_template_string(HTML_TEMPLATE, count=count,
version=version)

@app.route("/api")
def api():
```

```
    count = redis_client.get("hits") or 0
    return jsonify(visit_count=int(count),
version=os.getenv("APP_VERSION", "v1")), 200

@app.route("/health")
def health():
    try:
        redis_client.ping()
        return jsonify({"status": "ok"}), 200
    except redis.RedisError as exc:
        return jsonify(
            status="failed",
            detail=str(exc)
        ), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

2.1.2. Backend

Хранилище представлено в виде `Redis`. Для локального стенда пароль берется как переменное окружения, а порт хранилища не открывается наружу. Данные находятся в именованном объёме `volume redis-data`.

2.2. Ход работы

2.2.1. Контейнеризация

Для Python-сервиса собран образ на базе `python:3.13-slim`. Переменные окружения задают адрес `Redis` и параметры запуска `Flask`. Зависимости ставятся отдельно.

```
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    FLASK_APP=app.py \
    FLASK_RUN_HOST=0.0.0.0 \
    FLASK_RUN_PORT=5000

WORKDIR /app

COPY app/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./

EXPOSE 5000
```

```
CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]
```

Список зависимостей приложения в `requirements.txt`:

```
Flask==3.1.2
redis==7.1.0
```

2.2.2. Docker Compose

Файл `docker-compose.yml` описывает два сервиса: `frontend` и `redis`. Пароль `Redis` и остальные переменные находятся в `.env`. Порт `Redis` наружу не открывается, клиентская часть приложения доступен на `localhost:5002`.

```
services:
  frontend:
    build: .
    image: lab1-frontend:latest
    ports:
      - "5002:5000"
    env_file: .env
    environment:
      - REDIS_HOST=${REDIS_HOST:-redis}
      - REDIS_PORT=${REDIS_PORT:-6379}
      - REDIS_DB=${REDIS_DB:-0}
      - REDIS_PASSWORD=${REDIS_PASSWORD}
    depends_on:
      - redis
    deploy:
      replicas: 1
      restart_policy:
        condition: any

  redis:
    image: redis:7-alpine
    env_file: .env
    command: ["redis-server", "--appendonly", "yes", "--
requirepass", "${REDIS_PASSWORD}"]
    volumes:
      - redis-data:/data
    deploy:
      replicas: 1
      restart_policy:
        condition: any

volumes:
  redis-data:
```

Пример настройки окружения (в репозитории хранится как `.env.example`):

```
REDIS_PASSWORD=password
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_DB=0
```

Локальный запуск и проверка (до перехода в `Swarm`), как показано на рисунках 1-3:

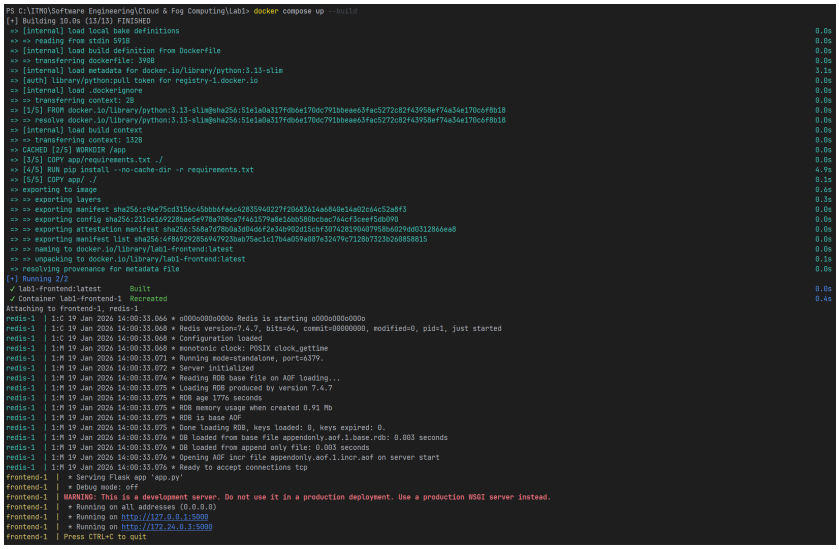

Рисунок 1 — Запуск команды `docker compose up --build`

```
curl -s http://localhost:5002/
```

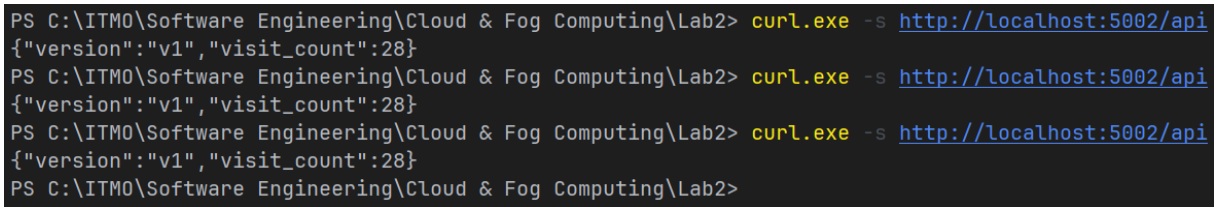

Рисунок 2 — отправка запросов через `curl`

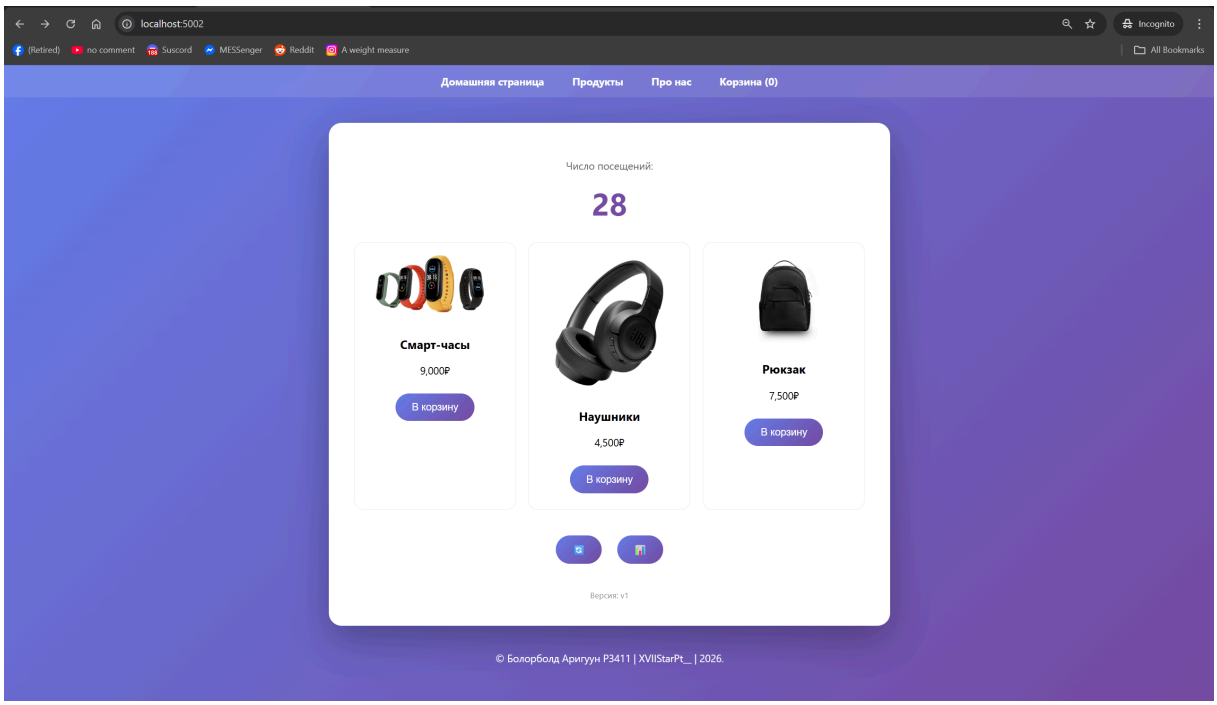

Рисунок 3 — отправка запросов через фронт

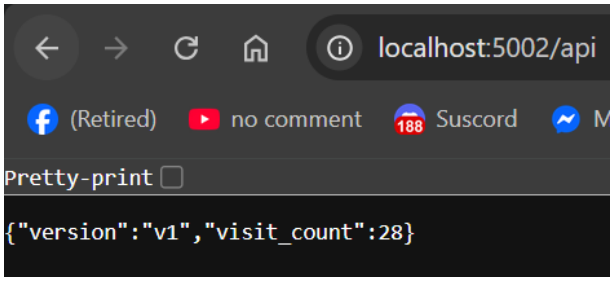

Рисунок 4 — отправка запросов через браузер

2.2.3. Инициализация `Swarm`

Перевод `Docker`-а в режим оркестрации. Команда выводит токены для присоединения узлов, как показано на рисунке 5:

```
docker swarm init
```

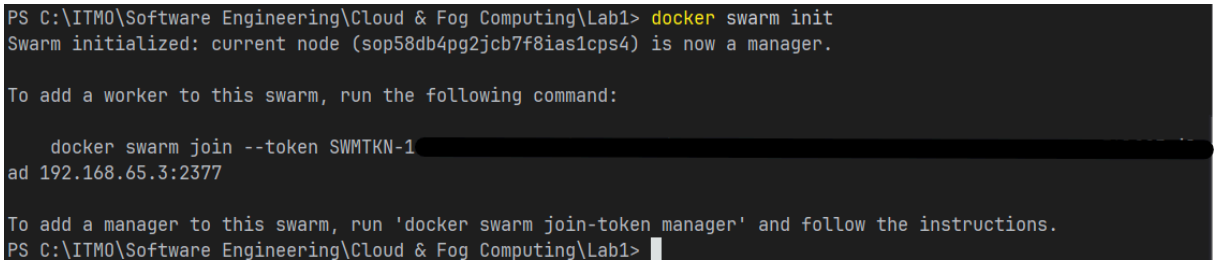

Рисунок 5 — Запуск `docker swarm`

2.2.4. Развертывание стека

Перед деплоем образ клиента собирается и тегируется, затем запускается стек `app`. Список сервисов показывает доступный порт фронта, как показано на рисунке 6.

```
docker compose build frontend
docker stack deploy -c docker-compose.yml app
docker stack services app
```

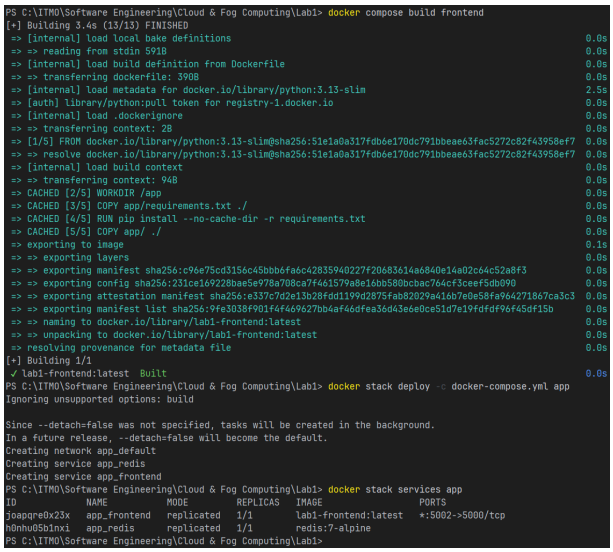

Рисунок 6 — Развертывание стека

2.2.5. Тест на отказ

Одна из реплик клиента намеренно завершается, после чего планировщик автоматически поднимает новую задачу, что видно по списку задач сервиса на рисунке 7.

```
docker ps --filter
"label=com.docker.swarm.service.name=app_frontend"
docker kill app_frontend.1.x11b6mxl630d6nfa1orleslji
docker service ps app_frontend
```

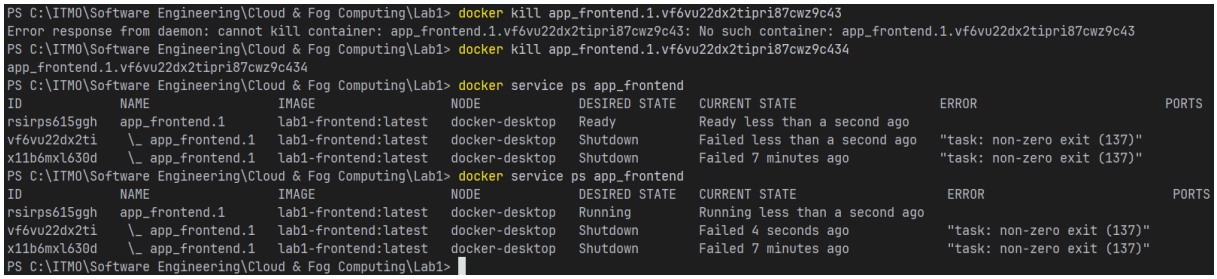

Рисунок 7 — Тест на отказ

2.2.6. Масштабирование

Масштабирование выполняется командой `docker service scale`. После этого у сервиса стало 5 реплик, как показано на рисунке 8.

```
docker service scale app_frontend=5
```

```
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab1> docker service scale app_frontend=5
app_frontend scaled to 5
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service app_frontend converged
PS C:\ITMO\Software Engineering\Cloud & Fog Computing\Lab1>
```

Рисунок 8 — Масштабирование сервиса

3. Вывод

В рамках лабораторного проекта было создано и запущено веб-приложение `Flask`, работающее в контейнерах и использующее `Redis` как базу данных. Изначально проект был сконфигурирован для локальной разработки с помощью `Docker Compose`, а затем эта же конфигурация успешно использовалась для развертывания в кластере `Docker Swarm`.
В процессе выполнения работы были продемонстрированы основные функции оркестратора:

- Отказоустойчивость: Автоматическое восстановление сервисов после сбоев.
- Горизонтальное масштабирование: Увеличение числа запущенных экземпляров сервиса `Flask` (до пяти реплик).

Все конфигурационные параметры хранятся централизованно в файле `.env`, а доступ к базе данных `Redis` ограничен внутренней сетью.
Полученный результат — опыт миграции приложения из среды разработки с одиночными контейнерами в полноценную, управляемую оркестратором инфраструктуру.